

Documentazione Progetto Sistemi Operativi: PandOSsh Fase 2

Luca Argentino, Milo Disalvatore, Matteo Mazzetti, Zeno Maccari

17 maggio 2026

1 Inizializzazione (`initial.c`)

Il modulo `initial.c` definisce il punto di ingresso del sistema e si occupa dell'inizializzazione generale del Nucleus.

1.1 Strutture Dati Globali

Le variabili globali definite nel modulo sono:

- `tod_start`: Memorizza il tempo di inizio per calcolare l'utilizzo della CPU.
- `readyQueue`: Coda dei processi in stato "ready".
- `processCount` e `softBlock_count`: Contatori rispettivamente dei processi attivi e di quelli bloccati in attesa di I/O o timer.
- `currProc`: Puntatore al PCB del processo attualmente in esecuzione.
- `device_semaphores`: Array dei semafori per sincronizzare i dispositivi esterni e lo Pseudo-clock.

1.2 Descrizione delle Funzioni

1.2.1 `pcb_t* allocFirstPcb()`

1. Alloca un nuovo PCB e ne inizializza lo stato del processore e imposta il Program Counter alla funzione `test`.
2. Inizializza a NULL o zero i restanti campi gestionali (albero dei processi, tempo di CPU, puntatore di supporto e semaforo bloccante).
3. Imposta lo Stack Pointer per utilizzare l'ultimo frame disponibile della RAM e restituisce il PCB.

1.2.2 void initVector()

1. Accede al Pass Up Vector del Processore 0 per configurare la gestione delle eccezioni.
2. Registra gli indirizzi dei due gestori principali: `uTLB_RefillHandler` per i TLB-Refill e `exceptionHandler` per tutte le altre eccezioni.
3. Imposta i rispettivi Stack Pointer al top della pagina di stack riservata al Nucleus.

1.2.3 int main()

1. Inizializza le strutture dati di base (`readyQueue`, PCB e ASL) e configura il Pass Up Vector richiamando `initVector()`.
2. Carica l'Interval Timer di sistema con un valore di 100 millisecondi.
3. Istanza il processo iniziale tramite `allocFirstPcb()`, lo inserisce nella `readyQueue` e incrementa il contatore dei processi.
4. Cede il controllo al sistema invocando lo scheduler tramite `dispatch()`.

2 Scheduler (scheduler.c)

Il modulo `scheduler.c` implementa il gestore dei processi tramite un algoritmo di scheduling round-robin con prelazione (time slice di 5ms), garantendo che ogni processo pronto abbia l'opportunità di essere eseguito.

2.1 Descrizione delle Funzioni

2.1.1 void dispatch()

1. Verifica lo stato della `readyQueue`. Se è vuota:
 - Arresta il sistema (`HALT()`) se non ci sono processi attivi (`processCount == 0`).
 - Entra in stato di attesa (`WAIT()`), abilitando gli interrupt, se ci sono processi bloccati (`softBlock_count > 0`).
 - Dichiarare un deadlock (`PANIC()`) se ci sono processi attivi ma nessuno è in attesa di I/O o timer.
2. Se la coda contiene processi, ne estrae il primo e lo imposta come processo corrente (`currProc`).
3. Carica il Processor Local Timer (PLT) con il time slice di 5 millisecondi.
4. Avvia l'esecuzione del processo caricandone lo stato sul processore (`LDST`).

3 Gestione Interrupt (`interrupts.c`)

Il modulo `interrupts.c` gestisce le eccezioni di interrupt sia dei timer (Processor Local Timer e Interval Timer) sia dei dispositivi hardware. I dispositivi gestiti sono Disk devices, Flash devices, Network devices, Printer devices e Terminal devices (che possono essere di tipo transmitter o receiver).

3.1 Descrizione delle Funzioni

3.1.1 `void pltInterruptHandler()`

1. Aumenta il PLT (timer locale del processore) di un quanto di 5ms.
2. Aggiorna il tempo di utilizzo del processore del processo corrente `currProc`.
3. Memorizza lo stato attuale del processore all'istante esatto in cui è avvenuto un interrupt e lo memorizza nel campo `p_s` del PCB del processo attuale.
4. Inserisce il processo interrotto nella `readyQueue`.
5. Invoca la funzione `dispatch()` dello scheduler.

3.1.2 `void itInterruptHandler()`

1. Ricarica l'Interval Timer con un nuovo time slice pari a 100ms.
2. Rilascia tutti i processi in attesa di un tick di timer e li inserisce nella `readyQueue`, diminuendo in numero di processi soft-bloccati (in attesa di eseguire un operazione I/O).
3. Resetta il valore dell'ultimo semaforo.
4. Salva lo stato attuale del processore nel momento in cui è avvenuto un interrupt e lo memorizza nel campo `p_s` del PCB del processo attuale.
5. Richiama l'unica procedura dello scheduler: `dispatch()`.

3.1.3 `void deviceInterruptHandler(int excCode)`

1. Mappa l'`excCode` che ottiene dal parametro a una delle righe corrispondenti agli interrupt hardware che possono essere:
 - Line 3: Disk
 - Line 4: Flash
 - Line 5: Ethernet
 - Line 6: Printer
 - Line 7: Terminal

2. Calcola l'indirizzo di memoria del device che genera l'interrupt ed esegue operazioni bit a bit per determinare quale dei vari device elencati sopra ha effettivamente generato l'interrupt.
3. Calcola l'indirizzo fisico di memoria dei registri di tale device in `devAddr` e ottiene l'indice del semaforo corrispondente.
4. Controlla se l'interrupt è generato da un terminale e in tal caso verifica se esso è di tipo transmitter o receiver, leggendo di conseguenza l'appropriato registro di stato.
5. Esegue un'operazione di tipo Verhoog sul semaforo di quel device: sbloccando il primo processo in attesa o facendo sì, se la coda è vuota, che il prossimo non si blocchi. Se un processo era in attesa di questo I/O:
 - Memorizziamo il codice di stato di quel processo nel registro `a0` che corrisponde a `p_s->gpr[24]`.
 - Decrementa il `softBlock_count`, ossia il contatore dei processi che aspettano un'I/O.
 - Restituisce il controllo al processo interrotto

Altrimenti viene richiamato lo scheduler.

3.1.4 void interruptHandler()

1. Esegue un AND bitwise tra l'`excState` e un'apposita maschera e determina la tipologia dell'interrupt.
2. Se era associato al Processor Local Timer richiama `pltInterruptHandler()`.
3. Se era associato all'Interval Timer invoca `itInterruptHandler()`.
4. Se invece era associato ad un device passa il controllo a `deviceInterruptHandler()`.

4 Eccezioni (exceptions.c)

Il modulo `exceptions.c` rappresenta il punto di ingresso per tutte le eccezioni (esclusi gli eventi di TLB-Refill) e si occupa di smistarle ai gestori appropriati. Implementa inoltre il meccanismo di "Pass Up or Die" per la gestione delle eccezioni a livello di supporto.

4.1 Descrizione delle Funzioni

4.1.1 void exceptionHandler()

1. Ottiene lo stato del processore al momento dell'eccezione tramite `getCurrExceptionState()` e ne estrae il codice (`cause`).

2. Verifica se l'eccezione è un interrupt (controllando il bit 31 del registro `cause`). In tal caso, passa il controllo a `interruptHandler()` e termina.
3. Se non è un interrupt, applica una maschera al codice dell'eccezione per determinarne la natura:
 - Se il codice è compreso tra 24 e 28, invoca `exc_tlbHandler()`.
 - Se il codice è 8 o 11 (SYSCALL), invoca `syscallHandler()`.
 - In tutti gli altri casi (Program Trap), invoca `exc_trapHandler()`.

4.1.2 void passUpOrDie(int exceptionType)

1. Verifica se il processo corrente possiede una struttura di supporto (`p_supportStruct == NULL`).
2. **Die:** Se la struttura non è presente, il processo corrente e tutta la sua progenie vengono terminati richiamando `nsys2(0, excState)`.
3. **Pass Up:** Se la struttura è presente, lo stato dell'eccezione salvato viene copiato nell'apposito campo `sup_exceptState` della struttura di supporto.
4. Recupera il contesto del livello di supporto.
5. Trasferisce il controllo al gestore del livello di supporto tramite l'istruzione LDCXT.

4.1.3 void exc_tlbHandler()

1. Gestisce le eccezioni del TLB richiamando semplicemente la funzione `passUpOrDie()` passandole come parametro la costante `PGFAULTEXCEPT`.

4.1.4 void exc_trapHandler()

1. Gestisce le eccezioni di tipo Program Trap richiamando la funzione `passUpOrDie()` passandole come parametro la costante `GENERALEXCEPT`.

4.1.5 void syscallHandler()

1. Estrae i parametri della System Call dai registri `a0`, `a1`, `a2` e `a3` dallo stato dell'eccezione.
2. Controlla se è stata richiesta una SYSCALL del Nucleus (valore negativo in `a0`) mentre il processore si trovava in User Mode (`MPP == 0`). In tal caso, simula una Program Trap modificando la causa in `PRIVINSTR` e richiama `exc_trapHandler()`.
3. Se la richiesta è valida, incrementa il Program Counter (`pc_epc`) di 4 byte per le chiamate del Nucleus (da -1 a -10) per evitare loop infiniti.

4. Utilizza uno **switch** per smistare l'esecuzione alla corrispondente funzione **nsysN()** in base al valore di **a0**. Se il codice della syscall non è riconosciuto, invoca **exc_trapHandler()**.

5 Chiamate di Sistema (syscall.c)

Il modulo **syscall.c** contiene l'implementazione delle 10 System Call fornite dal Nucleus (da **nsys1** a **nsys10**).

5.1 Descrizione delle Funzioni

5.1.1 `int nsys1(int a1, int a2, int a3)`

1. **CreateProcess**: Alloca un nuovo PCB. Se non ci sono PCB disponibili, restituisce **NOPROC** (-1).
2. Se l'allocazione ha successo, inizializza il nuovo processo copiando lo stato del processore dal parametro **a1**, imposta la priorità (**a2**) e il puntatore alla struttura di supporto (**a3**).
3. Inserisce il nuovo PCB nella **readyQueue** e lo aggiunge come figlio del processo corrente.
4. Incrementa il contatore dei processi (**processCount**) e restituisce il PID del nuovo processo.

5.1.2 `void nsys2(int a1, state_t *excState)`

1. **TerminateProcess**: Individua il processo da terminare. Se **a1** è 0, il bersaglio è il processo corrente; altrimenti, ricerca il PCB corrispondente al PID fornito attraversando alberi dei processi, **readyQueue** e code dei semafori.
2. Se il processo non viene trovato (e **a1** $\neq$ 0), imposta **a0** a **NOPROC** e restituisce il controllo.
3. Richiama la funzione ricorsiva **terminateProcess()** per eliminare il processo e tutta la sua progenie, rimuovendoli da eventuali code (semafori o ready queue) e decrementando i relativi contatori (incluso **softBlock_count** se bloccati su I/O).
4. Invoca lo scheduler tramite **dispatch()**.

5.1.3 void nsys3(int a1, state_t *excState)

1. **Passeren (P)**: Esegue un'operazione di wait sul semaforo il cui indirizzo fisico è passato in **a1**.
2. Se il valore del semaforo è maggiore di 0, lo decrementa e restituisce il controllo al processo chiamante.
3. Se il semaforo è ≤ 0 , salva lo stato del processore nel PCB, calcola e aggiorna il tempo di CPU consumato fino a quel momento (**p_time**).
4. Blocca il processo inserendolo nella coda del semaforo (**ASL**), scollega il processo corrente (**currProc** = **NULL**) e invoca **dispatch()**.

5.1.4 void nsys4(int a1, state_t *excState)

1. **Verhogen (V)**: Esegue un'operazione di signal sul semaforo all'indirizzo **a1**.
2. Tenta di sbloccare un processo in attesa su tale semaforo rimuovendolo dalla coda.
3. Se trova un processo bloccato, lo reinserisce nella **readyQueue**. Altrimenti, incrementa il valore del semaforo.
4. L'operazione non blocca mai il processo chiamante, che continua la sua esecuzione.

5.1.5 void nsys5(int a1, int a2, state_t *excState)

1. **DoIO**: Avvia un'operazione di I/O sincrona e blocca il processo corrente.
2. Calcola l'indice del semaforo corretto partendo dall'indirizzo fisico del registro di comando del device (**a1**), discriminando il tipo di dispositivo e gestendo in modo speciale i terminali (che hanno semafori separati per ricezione e trasmissione).
3. Scrive il comando (**a2**) nel registro del device, avviando fisicamente l'operazione.
4. Salva lo stato, aggiorna il tempo di CPU (**p_time**), incrementa il **softBlock_count** e blocca il processo sul semaforo del device appena calcolato.
5. Invoca lo scheduler tramite **dispatch()**.

5.1.6 void nsys6(state_t *excState)

1. **GetCPUTime**: Calcola il tempo di processore accumulato dal processo corrente sommando il valore salvato in **p_time** al tempo trascorso dall'inizio dell'attuale quanto di esecuzione (**tod_start**).
2. Inserisce il risultato nel registro **a0** dello stato dell'eccezione.

5.1.7 void nsys7(state_t *excState)

1. **WaitForClock:** Blocca il processo corrente fino al prossimo scatto del pseudo-clock (gestito dall'Interval Timer ogni 100ms).
2. Salva lo stato e aggiorna il tempo di esecuzione come nelle altre syscall bloccanti.
3. Incrementa il `softBlock_count`, inserisce il processo in coda al semaforo dedicato (`device_semaphores[NSUPPSEM]`) e invoca `dispatch()`.

5.1.8 void nsys8(state_t *excState)

1. **GetSupportPtr:** Restituisce il puntatore alla struttura di supporto del processo corrente inserendolo nel registro `a0`. Se il processo non ha una struttura di supporto, restituisce `NULL`.

5.1.9 void nsys9(int a1, state_t *excState)

1. **GetProcessID:** Se `a1` vale 0, inserisce in `a0` il PID del processo corrente.
2. Se `a1` è diverso da 0, restituisce il PID del processo padre. Se il processo non ha un padre (`p_parent == NULL`), restituisce 0.

5.1.10 void nsys10(state_t *excState)

1. **Yield:** Permette al processo corrente di rilasciare volontariamente il processore.
2. Salva lo stato del processore nel PCB e calcola il tempo di CPU consumato fino a quel momento.
3. Reinserisce il processo corrente all'interno della `readyQueue`.
4. Imposta `currProc` a `NULL` e invoca `dispatch()` per programmare il processo successivo.